Lab Session 06

Introduction and Working of Design Pattern in Software Construction

Objective

To understand the concept, purpose, and working of design patterns and apply them to real software scenarios. Students will implement and analyze three commonly used design patterns to improve software structure, flexibility, and maintainability.

Introduction

In real-world software development, developers frequently encounter similar design problems. As software systems grow larger and more complex, managing code becomes difficult. Without proper design, systems become:

- Hard to modify
- Difficult to test
- Error-prone
- Rigid and tightly coupled

For example:

- A system may depend on too many classes
- One change may break many modules
- Code duplication increases
- Maintenance becomes expensive

To solve such recurring problems, software experts introduced **Design Patterns**.

What are Design Patterns?

A design pattern is a **general reusable solution to a common software design problem**.

It is not a finished code but a **template** that guides developers in structuring code.

Design patterns help in:

- Writing clean code
- Reducing coupling
- Increasing reusability
- Improving scalability
- Making communication easier among developers

Why Design Patterns are Important

Design patterns are important because they:

1) Promote Reusability

Instead of solving the same problem from scratch, we reuse proven solutions.

2) Improve Communication

If a developer says “We are using Singleton,” others immediately understand the design.

3) Reduce Development Risk

Patterns are tested and widely used.

4) Improve Maintainability

Code becomes easier to modify and extend.

5) Provide Best Practices

They represent industry-standard solutions.

Categories of Design Patterns

Design patterns are generally divided into:

1. Creational Patterns

Deal with object creation. ***Example***: Singleton

2. Structural Patterns

Deal with object relationships. ***Example***: Facade

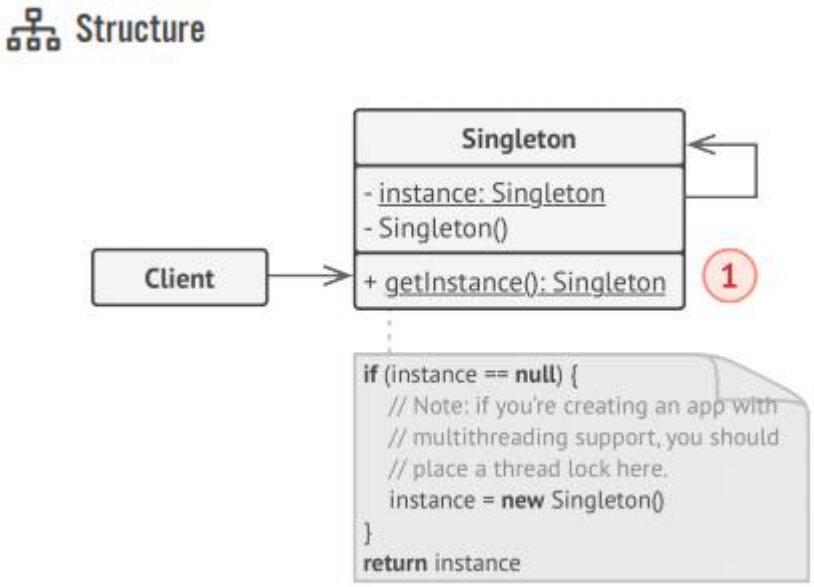
3. Behavioral Patterns

Deal with communication between objects. ***Example:*** Observer

In this lab, we study one from each category.

1. Singleton Design Pattern

Singleton is a creational design pattern that lets you ensure that a class has only one instance, while providing a global access point to this instance



In many software systems, some resources must be shared across the entire application. Creating multiple objects for such resources can cause:

- Data inconsistency
- Resource conflicts
- Increased memory usage
- Synchronization problems

Singleton solves this by guaranteeing that only one object exists.

Real-World Scenario

Consider a **Database Connection Manager**.

In a large system:

- Many modules need database access
- Creating a new connection each time is costly
- Too many connections may overload the database

So, we maintain **one shared database connection**.

This is a perfect use case for Singleton.

How Singleton Works

1. Constructor is **private**
   → prevents new keyword use
2. Static instance variable stores object
3. Public static getInstance() method
   → controls object creation
4. First call creates object
   → later calls return same object

Code Example

```
class Database {

    private static Database instance;

    // Private constructor
    private Database() {
        System.out.println("Database connected");
    }

    // Thread-safe getInstance
    public static Database getInstance() {

        if(instance == null) {
            synchronized(Database.class) {
                if(instance == null) {
                    instance = new Database();
                }
            }
        }
        return instance;
    }

    // Business method
    public void query(String sql) {
        System.out.println("Executing: " + sql);
    }
}
```

Client Code

```
class Application {
    public static void main(String[] args) {

        Database db1 = Database.getInstance();
        db1.query("SELECT * FROM users");

        Database db2 = Database.getInstance();
        db2.query("SELECT * FROM orders");

        // db1 and db2 refer to SAME object
    }
}
```

Explanation

1. Private Constructor

Prevents direct object creation.

2. Static Instance

Stores single object.

3. getInstance()

Creates object once, then reuses it.

4. Thread Lock

Prevents multiple threads from creating multiple instances.

Applicability

Use Singleton when:

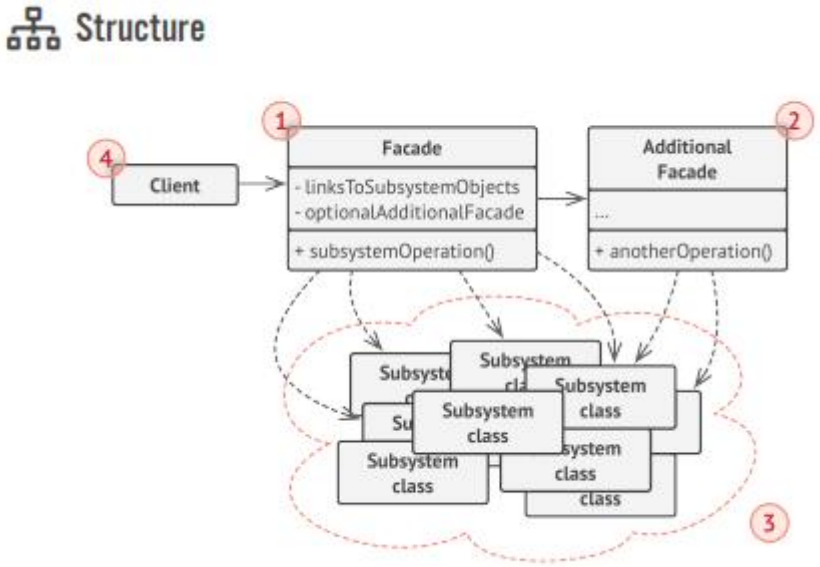
✔ Only one instance is needed
✔ Shared resource management required
✔ Strict control over global access needed

Examples:

- Database connection
- Logger system
- Configuration manager
- Cache manager

2. Facade Design Pattern

Facade is a structural design pattern that provides a simplified interface to a library, a framework, or any other complex set of Classes



The Facade pattern provides a **simplified interface to a complex subsystem**.

Large systems often consist of many classes that interact in complicated ways. Clients that use these systems must understand many internal details, which increases:

- Complexity
- Learning time
- Dependency on internal classes
- Maintenance difficulty

The Facade pattern hides this complexity by offering a single entry point.

Real-World Scenario

Consider a **video conversion system** using a third-party framework.

The framework contains many classes:

- VideoFile
- CodecFactory
- BitrateReader
- AudioMixer
- Multiple codecs

To convert a video, a developer must call these classes in the correct order.

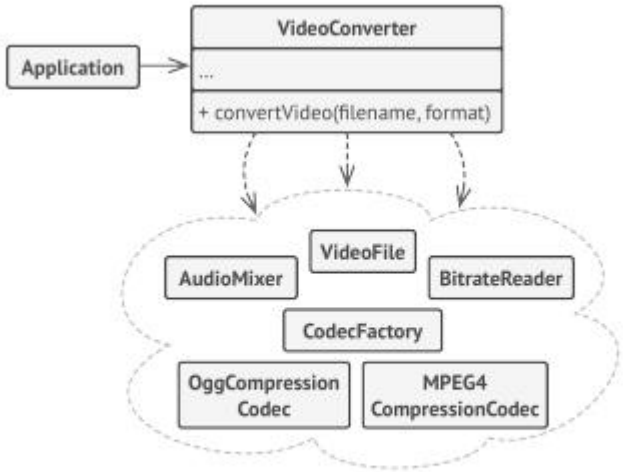
This makes the code:
❌ Hard to use
❌ Hard to update
❌ Strongly coupled to framework

Facade solves this by wrapping everything into one class.

How Facade Works

1. A Facade class is created
2. It calls subsystem classes internally
3. Client interacts only with Facade
4. Subsystem complexity is hidden

Scenario based Diagram



An example of isolating multiple dependencies within a single facade class.

Code Example

```
class VideoFile {}
class OggCompressionCodec {}
class MPEG4CompressionCodec {}

class CodecFactory {
    static Object extract(VideoFile file) {
        return new OggCompressionCodec();
    }
}

class BitrateReader {
    static String read(String filename, Object codec) {
        return "buffer";
    }
```

```
class BitrateReader {
    static String read(String filename, Object codec) {
        return "buffer";
    }

    static String convert(String buffer, Object codec) {
        return "converted";
    }
}

class AudioMixer {
    String fix(String result) {
        return result + " fixed";
    }
}
```

```
// FACADE
class VideoConverter {
    public File convert(String filename, String format) {

        VideoFile file = new VideoFile();
        Object sourceCodec = CodecFactory.extract(file);

        Object destinationCodec;
        if(format.equals("mp4"))
            destinationCodec = new MPEG4CompressionCodec();
        else
            destinationCodec = new OggCompressionCodec();

        String buffer = BitrateReader.read(filename, sourceCodec);
        String result = BitrateReader.convert(buffer, destinationCodec);
        result = new AudioMixer().fix(result);

            return new File(result);
      }
}
```

Client Code

```
java

class Application {
    public static void main(String[] args) {
        VideoConverter converter = new VideoConverter();
        File mp4 = converter.convert("video.ogg","mp4");
    }
}
```

Explanation

✔ Client does not interact with codecs
✔ All complexity handled by Facade
✔ Framework can be replaced easily

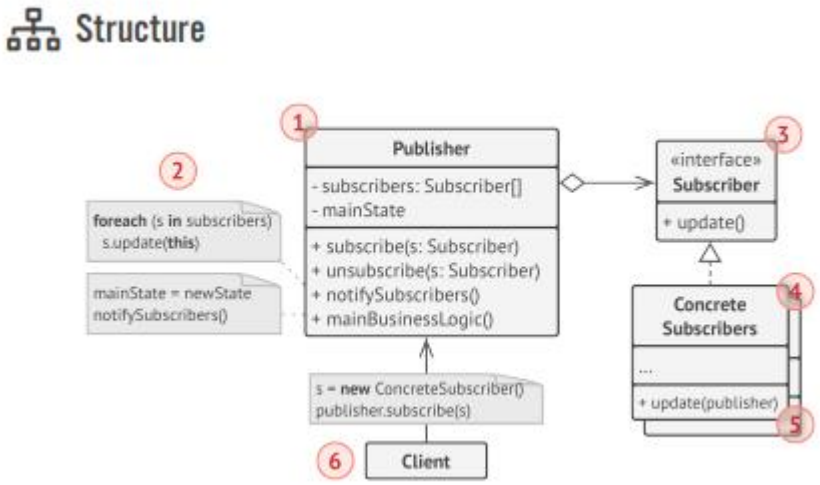
Applicability

Use Facade when:

✔ System is complex
✔ Many classes must be used together
✔ You want layered architecture
✔ You want to reduce coupling

3. Observer Pattern

Observer is a behavioral design pattern that lets you define a subscription mechanism to notify multiple objects about any events that happen to the object they're observing.



The Observer pattern allows objects to be notified automatically when another object changes state.

It defines a **one-to-many relationship**:
One publisher → many subscribers

This is useful in event-driven systems.

Real-World Scenario

Consider a **text editor**.

When a file is:

- Opened
- Saved

The system must notify:

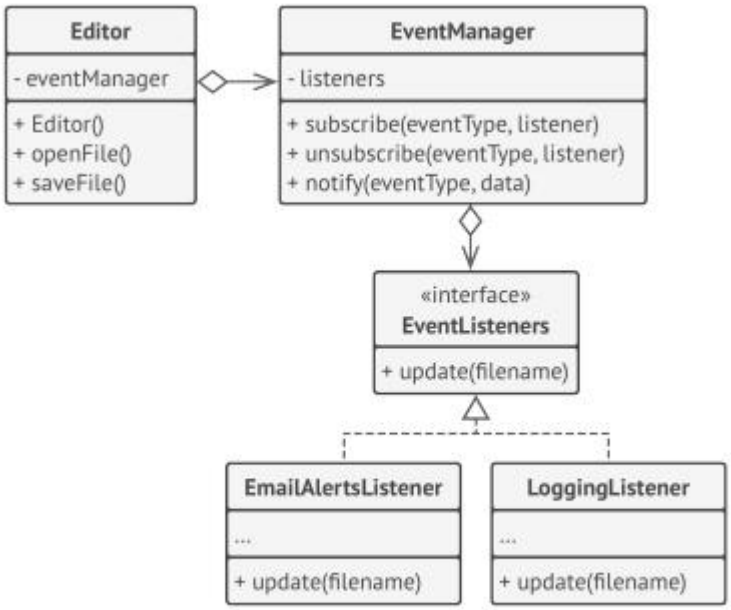
- Logging service
- Email alert system

Instead of hardcoding notifications, we use Observer.

How Observer Works

1. Publisher maintains subscriber list
2. Subscribers register/unregister
3. Publisher notifies on change
4. Subscribers react independently

Scenario based Diagram



Notifying objects about events that happen to other objects.

Code Example

```
interface EventListener {
    void update(String filename);
}

class EventManager {
    Map<String,List<EventListener>> listeners = new HashMap<>();

    void subscribe(String event, EventListener l){
        listeners.computeIfAbsent(event,k->new ArrayList<>()).add(l);
    }

    void notify(String event, String data){
        for(EventListener l:listeners.get(event))
            l.update(data);
    }
}

class Editor {
    EventManager events = new EventManager();
    String file;

    void openFile(String path){
        file = path;
        events.notify("open",file);
    }

    void saveFile(){
        events.notify("save",file);
    }
}
```

```
class LoggingListener implements EventListener{
    public void update(String filename){
        System.out.println("Log: "+filename);
    }
}

class EmailAlertsListener implements EventListener{
    public void update(String filename){
        System.out.println("Email alert: "+filename);
    }
}
```

Client Code

```java
class Application {
    public static void main(String[] args){

        Editor editor = new Editor();

        editor.events.subscribe("open",new LoggingListener());
        editor.events.subscribe("save",new EmailAlertsListener());

        editor.openFile("test.txt");
        editor.saveFile();
    }
}
```

Explanation

- ✓ The list of subscribers is compiled dynamically: objects can start or stop listening to notifications at runtime, depending on the desired behavior of your app.
- ✓ In this implementation, the editor class doesn't maintain the subscription list by itself. It delegates this job to the special helper object devoted to just that. You could upgrade that object to serve as a centralized event dispatcher, letting any object act as a publisher.
- ✓ Adding new subscribers to the program doesn't require changes to existing publisher classes, as long as they work with all subscribers through the same interface.

Applicability

Use Observer when:

✔ Event notification needed
✔ Many objects depend on one
✔ Loose coupling required
✔ Dynamic subscription needed

EXERCISE

Task 1 — Singleton Pattern

Scenario: University Examination System

A university is developing an **online examination system** used by thousands of students simultaneously.

The system includes:

- Student module
- Teacher module
- Result processing module
- Admin panel

All modules must access a **central Exam Configuration Manager** that stores:

- Exam duration
- Grading policy
- Passing criteria
- Exam rules

If multiple configuration objects are created:

- Rules may become inconsistent
- Different modules may follow different policies
- System integrity may break

Your Task

1. Explain why this scenario requires Singleton.
2. Design a Singleton class `ExamConfigManager`.
3. Ensure only one instance exists.
4. Show how different modules access the same instance.
5. Draw UML diagram.
6. Discuss what problem occurs if Singleton is not used.

Task 2 — Facade Pattern

Scenario: Smart City Service Portal

A city launches a **Smart City Portal** where citizens can request services like:

- Garbage collection
- Water supply complaint
- Streetlight repair
- Road maintenance

Each request requires interacting with multiple departments:

- Complaint Registration System
- Department Routing System
- SMS Notification System
- Tracking System
- Billing System

Currently, the client app directly interacts with all these subsystems, making the code complex.

Your Task

1. Identify problems in current system.
2. Design a `CityServiceFacade`.
3. Facade should provide simple methods like:
    - requestService()
    - trackRequest()
4. Show how facade hides complexity.
5. Draw UML diagram.
6. Explain how facade helps future upgrades.

Task 3 — Observer Pattern

Scenario: E-Learning Platform

An online learning platform wants to notify users when:

- A new lecture is uploaded
- Assignment deadline is near
- Grades are released

Subscribers include:

- Students
- Teachers
- Academic advisors
- Parents (optional subscription)

Each subscriber wants different notifications.

Your Task

1. Identify publisher and observers.
2. Design an Observer system.
3. Allow dynamic subscription/unsubscription.
4. Implement at least two subscriber types.

5. Draw UML diagram.
6. Explain what happens if Observer is not used.

Task 4 — Combined Design Challenge

Scenario: Online Event Management Platform

An event platform manages:

- Event registration
- Notifications
- Payment processing
- Event rules

Requirements:

- Only one Payment Gateway Config (Singleton)
- Simplified booking interface (Facade)
- Notify users of event updates (Observer)

Your Task

1. Identify where each pattern applies.
2. Design system architecture.
3. Draw UML diagram.
4. Provide sample code for one pattern.
5. Justify design decisions.